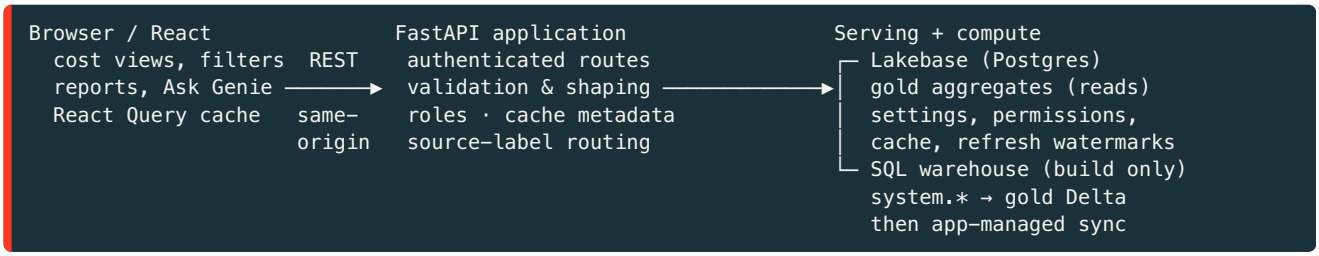
cost-obs — Architecture (v2.1)

A Databricks App that turns governed platform usage into interactive cost analysis. The browser calls a FastAPI service. **Normal dashboard tab reads are served from Lakebase.** A bound SQL warehouse reads Unity Catalog `system.*` tables only to build and refresh gold aggregates. The app then copies those aggregates into Lakebase with an app-managed sync — not managed synced tables. Static design content only — no customer, account, workspace, or user identifiers. · October 2026

Architecture at a glance



- **Authentication + governance:** Databricks Apps session → FastAPI role checks → app service principal for warehouse SQL and Lakebase → warehouse `CAN USE` + Unity Catalog grants + Lakebase `CAN_CONNECT_AND_CREATE`.
- **Scheduled refresh:** scheduler → service principal → incremental gold rebuild on the warehouse → app-managed range-replace into Lakebase → watermarks, cache invalidation.
- **On-demand refresh:** tab refresh clears that tab's cache only; administrator rebuild recreates gold and re-syncs Lakebase.
- **Ask Genie:** separate conversational surface. Calls run as the signed-in user against a curated Genie space. The Genie *tab* is static reporting, not chat.

Components

Component	Role
React browser interface	Cost views, filters, settings, Ask Genie drawer, customer-facing report downloads
FastAPI application	Authenticated API: request IDs, validation, Lakebase-first reads, gold refresh coordination
Lakebase (Postgres)	Default serving store for gold aggregates, app state, permissions, and response-cache metadata
Databricks SQL warehouse	Governed SQL execution plane used to *build and refresh* gold from `system.*`. Not queried on a normal tab read
Databricks system tables	Account-level billing, query, compute, Lakeflow, serving, and workspace metadata — warehouse-only at refresh time
App-managed gold sync	After each gold build, the app copies computed gold into Lakebase. No managed synced tables, no Unity Catalog catalog over Postgres
Genie space	Optional curated space for Ask Genie. Seeded by the setup job; not bound as an app resource
AI Gateway	Parent AI surface in the product. AI/ML cost views render inside this tab

Request & data flow

1. A signed-in user opens the React interface in Databricks Apps and selects dates, workspaces, or shared sources.
2. The browser sends same-origin requests to tab-specific FastAPI routes. Middleware assigns a request ID, records status and duration, and carries an optional source-label filter.
3. FastAPI serves a valid cached bundle from Lakebase when possible.
4. On a cache miss, dashboard reads query **Lakebase gold** (Postgres). The warehouse is not opened for that request.
5. FastAPI returns shaped JSON. React renders the dashboard.
6. Refresh and rebuild are the only paths that scan `system.*`. Those jobs write gold Delta tables, then the app range-replaces the same computed rows into Lakebase.

What lives where

Concern	Lives in	Accessed how
Dashboard gold (`daily_*` aggregates)	Lakebase (copied from warehouse gold)	App Postgres pool
App state, settings, permissions, cache metadata	Lakebase	App Postgres pool
Gold **build / refresh**	SQL warehouse over `system.*`	Setup job, scheduled refresh, admin rebuild
Source-label / shared-source gold	Lakebase (local + imported shared gold, labeled by source)	App Postgres pool
Ask Genie	Curated Genie space	User identity, not the app service principal
Connected cloud invoice actuals	**Out of scope**	Not a v2.1 serving path
Classic VM / cluster infrastructure	Computed estimate in Lakebase (`daily_infra_cluster_spend`)	App Postgres pool

Lakebase is the serving database, not a general OLAP replacement for `system.*`. Dimensions such as list prices and workspace names are denormalized into gold at build time so dashboard joins stay in Postgres.

Lakebase serving (default)

The app starts with Lakebase-first serving on (`COST_OBS_LAKEBASE_READ=on`).

- **Normal tab reads** hit Lakebase gold. There is no warehouse query on that path.
- If Lakebase is unreachable or required gold is absent, the API reports the data as temporarily unavailable rather than opening the warehouse on a dashboard request.
- `shadow` and `off` are migration diagnostics only. They are not customer serving modes and are not part of the supported steady-state architecture.
- Per-query SQL detail (`dbsql_cost_per_query`) is warehouse-built and **not** copied into Lakebase. It is too large for serving gold and is not on the normal tab path.

App-managed gold sync

Managed Databricks synced tables are not used. A synced table would require a Unity Catalog catalog registered over Lakebase, which a typical app service principal cannot create.

Instead, after each gold build:

1. The warehouse incrementally `MERGE`s recent `system.*` partitions into gold Delta tables.
2. Under the same write guard, the app reads those rows and **range-replaces** them in Lakebase (`DELETE` the reprocess window, then batch insert).
3. A full backfill replaces the whole table. Incremental syncs are idempotent and cannot duplicate.
4. Watermarks record build time and Lakebase sync time separately so a stalled build cannot hide behind a later sync (or the reverse).

Account-level rows with a null workspace id are stored with a sentinel key so Postgres primary keys stay valid.

App state in Lakebase

Settings, roles, and cache metadata are native Lakebase tables — not warehouse queries on the request path.

Lakebase table	Purpose
`app_config`	Catalog/schema, schedule, pricing, workspace filter, source-share registry, unified-view list
`user_permissions`	Administrator / viewer role overrides
`response_cache`	Shared dashboard-bundle cache (TTL enforced)
`alerts / alert_events`	Threshold alerts and fire history
`tagging_rules`	Customer tagging rules
`workspace_cache`	Workspace display-name cache
`setup_state` / `build_progress`	Setup-job markers and rebuild progress
Sync watermark table	Per-aggregate last sync, window, and row counts

Some values are dual-written to Delta or a local file for migration and recovery. Lakebase remains the serving source of truth.

Source sharing, unified routing, and capability

Administrators can add additional gold sources — typically Delta-shared catalog/schema pairs that match this app's gold table shapes.

- During refresh, compatible shared gold is validated, labeled with `source_label`, and copied into the same Lakebase serving shape as local gold.
- Local data is always included and labeled as the local source.
- The browser can pass `source_labels` to narrow a request. That filter is applied in Lakebase.
- **Source capability:** if the selected shares do not publish the gold tables a view needs, the API returns a typed unavailable payload instead of an empty or invented chart.
- Warehouse-side unified views can be used as refresh inputs, but they are never queried by a dashboard request. Serving copies retain `source_label` in Lakebase.

Source-share configuration itself is stored in Lakebase.

Genie, Ask Genie, and AI Gateway

Genie tab — static cost reporting (not chat). Charged vs free-tier Genie spend, workspace grain, and Code / One / Agents / Other surface splits. Served from Lakebase gold built from billing usage and list prices.

Ask Genie — a slide-out conversational drawer, separate from the Genie tab. Status / message / reply wrap the Genie conversation API. Calls run as the signed-in user so Unity Catalog and space permissions apply. The space is curated over billing usage, list prices, optional account prices, and workspace names. Unsupported questions are rejected rather than guessed. Cloud invoice actuals and query-level warehouse cost are not in the curated space.

AI Gateway tab — the parent AI surface. AI/ML cost views (serving endpoints, ML clusters, agents, models) render inside this tab; there is no standalone AI/ML tab. Optional AI Gateway token/route analytics require an account-admin–managed derived table over `system.ai_gateway.usage`. The setup job does not create that table. When the table is present, gold `daily_ai_gateway` is built and synced to Lakebase.

Cloud costs: estimates only

Connected AWS / Azure / GCP **invoice actuals are out of scope** for this architecture. The Cloud Costs view shows a **computed VM / cluster infrastructure estimate** from cluster metadata and billing quantities, stored in Lakebase as `daily_infra_cluster_spend`. Databricks DBUs are not treated as VM node-hours.

Tab-by-tab serving

Tab	What the user sees	Lakebase gold (normal read)
$DBU Spend	Account spend, product / SKU, jobs & pipelines, interactive compute	`daily_usage_summary`, `daily_product_breakdown`, `daily_workspace_breakdown`, `daily_sku_breakdown`, `daily_job_pipeline_spend`, `daily_interactive_spend`
SQL	SQL vs Genie attribution, warehouse and source breakdowns	`sql_tool_attribution`, `daily_dbsql_source`, `daily_query_stats`, `daily_dbsql_warehouse_type`, `daily_dbsql_top`
AI Gateway	AI Gateway plus AI/ML cost views	`daily_ai_gateway` (when configured), `daily_aiml_endpoints`, `daily_aiml_models`, `daily_aiml_ml_clusters`, `daily_aiml_agents`
Genie	Static Genie charged / free-tier / surface split	`daily_sku_breakdown` (Genie surfaces and charge class)
Apps	Databricks Apps spend	`daily_apps_summary`
Tagging	Tag pairs and coverage	`daily_tag_summary`, `daily_tag_coverage`, `daily_untagged_objects`
Users	Identity spend	`daily_user_spend`
KPIs & Trends	Platform KPIs and trends	`daily_usage_summary`, `daily_query_stats`, `daily_workspace_breakdown`
Cloud Costs	Estimated classic VM / cluster infrastructure	`daily_infra_cluster_spend`
Optimize	Warehouse health and rightsizing findings	`warehouse_optimizer_findings`, `daily_dbsql_region_scope`

Optional use-case tracking is a settings-gated tab and uses the same Lakebase-first pattern when enabled.

Deployment lifecycle

All infrastructure is declared in the Databricks Asset Bundle: the app, SQL warehouse binding, Lakebase project (default production branch and primary endpoint), setup job, and environment injection.

```
databricks bundle deploy -t <target>
databricks bundle run cost_obs_setup -t <target>
```

`bundle deploy` creates or updates the app, Lakebase project, bindings, and job. It does not run grants that cannot be issued at plan time.

`cost_obs_setup` is the one-time (and repairable) onboarding job. Tasks are idempotent:

1. **validate** — warehouse, Lakebase endpoint, catalog/schema, deployment mode.
2. **bootstrap_lakebase** — schema, OAuth role, grants, default privileges, `setup_state`.
3. **migrate_delta** — on a v1 upgrade, copy legacy app state into Lakebase. No-op on a fresh install. Does not build gold.
4. **build_gold** — create/refresh warehouse gold from `system.*` (full on fresh, incremental on upgrade) and copy dashboard aggregates into Lakebase so the app does not wait on a Settings rebuild.
5. **seed_genie** — create or reuse the customer Genie space, grant the app service principal run access, store the id in Lakebase.
6. **setup_usage_policy** — serverless usage-policy attribution when available.
7. **verify** — smoke checks and the final setup marker.

Two motions are supported: **fresh** (no prior cost-obs) and **migrate** (keep an existing v1 catalog, schema, and warehouse). Pin the motion in the target file. `auto` probes for existing gold and is only a fallback.

The Genie space is **not** an app-resource binding. An empty space id is rejected by the Apps API and would break deploy. The app reads `GENIE_SPACE_ID` from the environment when set, otherwise the id stored in Lakebase.

Databricks Apps installs Python dependencies, starts FastAPI (two workers), and serves the committed frontend artifact. It does not build the frontend at deploy time.

Refresh paths

Scheduled gold refresh — nightly by default (weekly or monthly selectable; hour configurable). One worker holds a lock → incremental `MERGE` of recent source partitions into gold → app-managed Lakebase sync → watermarks and history → response-cache invalidation → optional alert evaluation.

Administrator full rebuild — Settings rebuild: administrator check → locked background rebuild → gold recreated for the configured history window → full Lakebase copy → caches cleared.

On-demand tab refresh — clears that tab's in-process and shared response-cache entries and refetches. Gold tables are not rebuilt.

There is no warehouse keepalive. The warehouse warms only for setup, build, or refresh work. Health polling is REST-only and issues no SQL.

Billing system tables can lag by minutes to a few hours. The product does not claim real-time spend.

Security boundaries

- Databricks Apps authenticates the browser session and forwards identity to the application.
- FastAPI uses forwarded identity for **app roles**. Settings mutations, rebuilds, and permission changes require the administrator role. Viewers read dashboards only.

- The **app service principal** performs warehouse SQL (gold build and refresh) and Lakebase OAuth (serving reads, state writes, gold sync). Forwarded user tokens are not used for those paths.
- **Ask Genie** is the exception: conversation calls use the signed-in user's workspace client so space and catalog grants apply to that person.
- Unity Catalog privileges govern `system.*` and gold Delta access. The bound warehouse separately enforces `CAN USE`.
- Lakebase binding is `CAN_CONNECT_AND_CREATE`. Runtime grants (connect, default privileges) are applied by the setup job, not at Terraform plan time.
- Workspace scoping is enforced in the application. Lakebase is not exposed as a general SQL endpoint to dashboard users.
- No customer secrets, tokens, or account identifiers belong in this document or the generated PDF.

Resilience and recovery

Situation	Behavior
Lakebase available, gold present	Serve Lakebase. No warehouse query
Lakebase error or empty required gold	Return a typed temporary-unavailable response; do not query the warehouse
Lakebase not bound	Setup is incomplete; dashboard serving remains unavailable until repaired
Shared-source filter active	Filter source-labeled Lakebase gold
Selected sources lack a required table	Typed "unavailable" response — no invented totals
Optional gold source missing	That table is skipped; the rest of the rebuild continues
One aggregate sync fails	Other tables still sync; status is recorded per table
Concurrent refresh	Write guard serializes gold build and Lakebase sync
App worker overlap	File lock so only one worker runs the scheduled rebuild

Source-table inventory (warehouse, refresh only)

Core analytic system tables: `system.billing.usage`, `system.billing.list_prices`, `system.query.history`, `system.compute.clusters`, `system.compute.warehouses`, `system.compute.warehouse_events`, `system.lakeflow.jobs`, `system.lakeflow.pipelines`, `system.lakeflow.job_run_timeline`, `system.serving.served_entities`, `system.access.workspaces_latest`

Optional: `system.billing.account_prices` when account pricing is enabled. `system.ai_gateway.usage` is not queried by the app; an administrator-managed derived table is required if AI Gateway token analytics are enabled.

Permission probe only: `system.access.audit` — readiness reporting, never an analytic input.

Gold aggregates copied into Lakebase (names are stable; the configured catalog/schema holds the warehouse originals): `daily_usage_summary`, `daily_product_breakdown`, `daily_workspace_breakdown`, `sql_tool_attribution`, `daily_query_stats`, `daily_tag_summary`, `daily_apps_summary`, `daily_sku_breakdown`, `daily_job_pipeline_spend`, `daily_interactive_spend`, `daily_tag_coverage`, `daily_dbsql_source`, `daily_dbsql_top`, `daily_dbsql_warehouse_type`, `daily_dbsql_region_scope`, `daily_aiml_endpoints`, `daily_aiml_models`, `daily_aiml_ml_clusters`, `daily_aiml_agents`, `daily_user_spend`, `daily_untagged_objects`, `daily_object_spend`, `daily_infra_cluster_spend`, `warehouse_optimizer_findings`, and `daily_ai_gateway` when configured. **Not copied:** `dbsql_cost_per_query` (per-query grain). **Out of scope:** connected cloud invoice actuals and any gold built only to serve those exports.